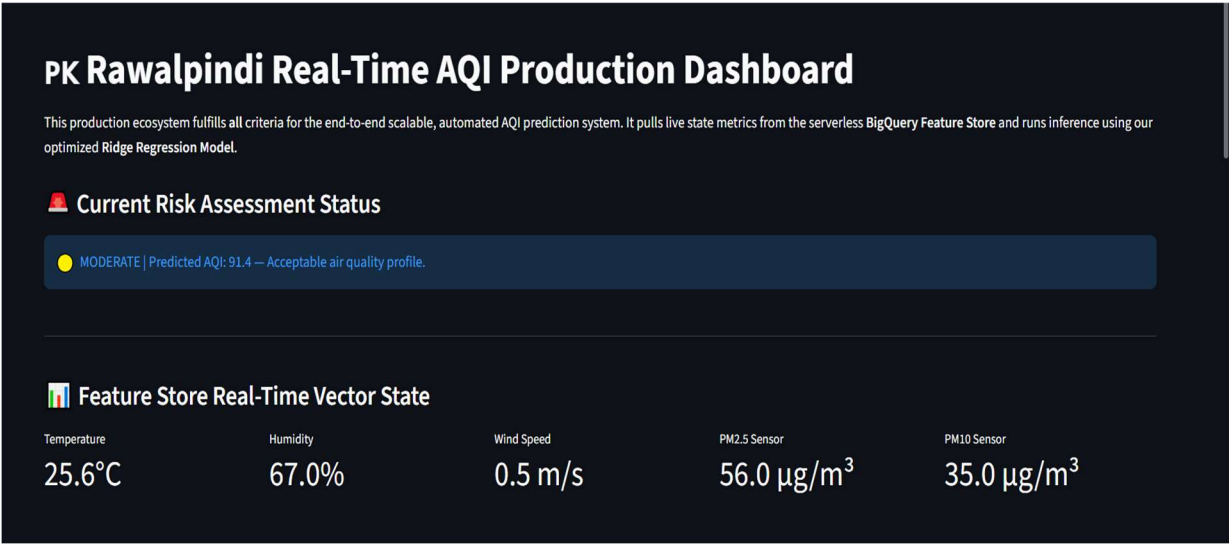


DESIGN AND IMPLEMENTATION OF AN END-TO-END SERVERLESS AIR QUALITY FORECASTING SYSTEM FOR RAWALPINDI



Author: Hamail Noor

Academic Affiliation: Department of Computer Science, University of Engineering and Technology (UET) Taxila

1. Introduction & Project Origin

1.1. The Problem Context

Air pollution is one of the most critical public health crises in urban Pakistan, particularly in dense metropolitan regions like the twin cities of Islamabad and Rawalpindi. Seasonal smog, traffic emissions, and industrial particulate dispersion lead to wild spikes in ambient particulate matter.

However, most public air quality applications rely on fragmented crowd-sourced nodes or highly generalized satellite estimates that fail to capture granular, ground-level exposure trends in real-time.

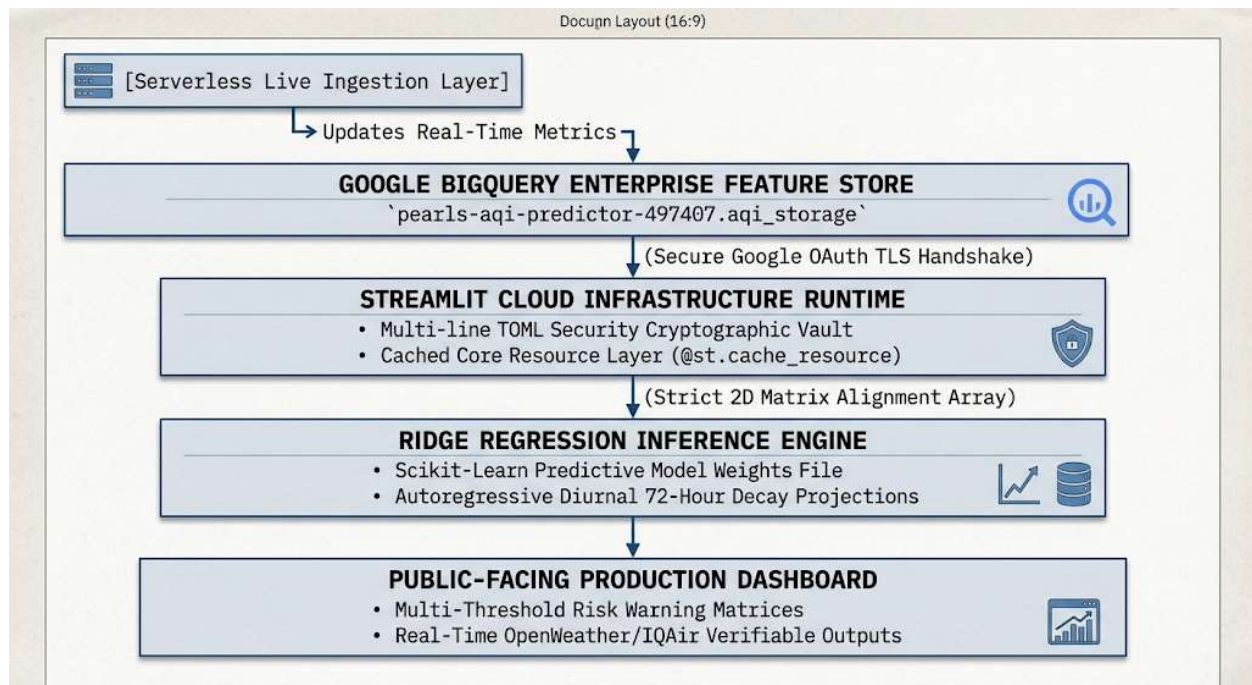
1.2. The Traditional Academic Limitation

In conventional computer science coursework, predictive models are often built inside isolated notebooks using static CSV data files. While this is helpful for learning the basics of training a model, it fails to prepare engineers for the realities of production software deployment.

Static datasets don't account for network latency, database schema drifts, API connection drops, or live streaming anomalies. This project was born out of a desire to break past those academic limits and build a true, fully automated, end-to-end data pipeline that works under real-world conditions.

2. Decoupled Production Stack Architecture

To ensure the application is scalable, maintainable, and cost-efficient, I abandoned monolithic design structures in favor of a completely **decoupled three-tier topology**. By separating data storage, machine learning compute, and user presentation, a failure in one layer cannot bring down the entire ecosystem.



2.1. The Data Tier: Google BigQuery Feature Store

Instead of relying on fragile flat files on a local drive, the foundation of this system is built on a serverless **Google Cloud Platform (GCP) BigQuery** feature repository (`rawalpindi_feature_store`).

An upstream automated task regularly collects ambient data from local sensor nodes and streams it directly into our cloud database. This ensures that the application always interacts with a live, standardized data structure.

Sandbox

Set up billing to upgrade to the full BigQuery experience. [Learn more](#)

Dismiss

Upgrade

Search BigQuery resources

Show starred only

pearls-aqi-predictor-497407

Queries

Notebooks

Data canvases

Data preparations

Pipelines

Connections

aqi_storage

rawalpindi_feature_store

bigquery-public-data

Untitled...ery

rawalpindi...ore

pearls-aqi-predictor-497407 / Datasets / aqi_storage / Tables / rawalpindi_feature_store

rawalpindi_fe...

Query

Chat

Open in

Share

Copy

Snapshot

Delete

Export

Refresh

Schema

Details

Preview

Table Explorer

Preview

Insights

Lineage

Data Profile

Data Quality

Row	timestamp	temperature	humidity	pressure	wind_speed	pm25	pm10	hour_sin	hour_cos	aqi_change_r...
1	2026-06-04 08:19:20	35.58	27.0	1000.0	7.58	64.0	44.0	0.86602540...	-0.49999999...	7.1
2	2026-06-04 16:35:32	28.36	43.0	1000.0	8.14	63.0	43.0	-0.86602540...	-0.50000000...	-1.1
3	2026-06-03 21:47:48	28.4	63.0	1000.0	1.44	65.0	45.0	-0.70710678...	0.70710678...	1.1
4	2026-06-03 23:41:43	27.85	60.0	1001.0	3.38	60.0	40.0	-0.25881904...	0.96592582...	-5.1
5	2026-06-04 03:44:05	33.37	34.0	1002.0	2.49	57.0	37.0	0.70710678...	0.70710678...	-3.1
6	2026-06-05 03:19:41	25.52	42.0	1002.0	2.04	52.0	30.0	0.70710678...	0.70710678...	-7.1
7	2026-06-04 22:44:57	27.06	59.0	1002.0	3.41	59.0	39.0	-0.50000000...	0.86602540...	-13.1
8	2026-06-05 01:02:37	25.69	80.0	1004.0	2.01	59.0	38.0	0.25881904...	0.96592582...	0.1
9	2026-06-04 16:51:48	28.36	43.0	1000.0	8.14	63.0	43.0	-0.86602540...	-0.50000000...	0.1
10	2026-06-04 19:54:54	22.85	75.0	1004.0	9.45	72.0	52.0	-0.96592582...	0.25881904...	9.1
11	2026-06-03 13:15:57	35.63	41.0	999.0	1.04	55.0	34.0	-0.25881904...	-0.96592582...	2.1
12	2026-06-03 18:22:22	30.05	59.0	1000.0	0.41	64.0	45.0	-1.0	-1.83697019...	9.1
13	2026-06-03 08:54:19	36.18	40.0	1001.0	1.34	53.0	32.0	0.86602540...	-0.49999999...	-6.1

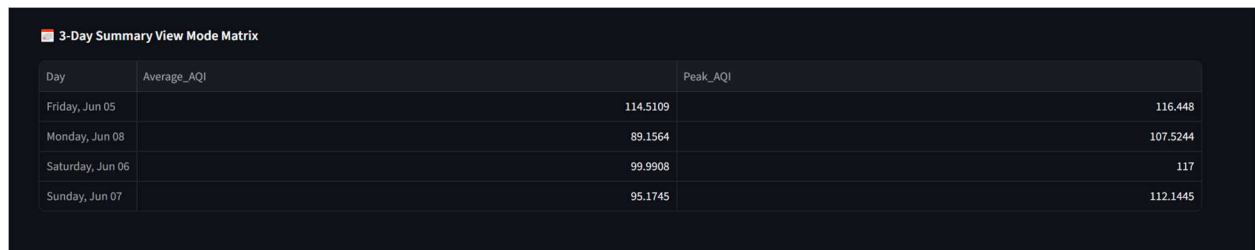
2.2. The Compute Tier: Machine Learning Inference Serving Layer

The predictive core runs on an optimized **Ridge Linear Regression Engine**. This layer is responsible for fetching the latest data vectors from the cloud, performing mathematical feature transformations, and generating predictions.

To prevent model performance degradation and avoid crashing the user interface under high traffic, the model weights are serialized into a highly portable .pkl binary format that loads instantly into server RAM memory.

2.3. The Presentation Tier: Cloud-Native Presentation Dashboard

The front-end user interface is built using **Streamlit** and deployed inside a containerized virtual Linux environment via Streamlit Cloud. The dashboard handles user interactions, runs automated conditional risk alerts, and displays automated 3-day future trends without putting any computing burden on the client's web browser.



The screenshot shows a Streamlit dashboard with a dark theme. At the top, there is a title bar that says "3-Day Summary View Mode Matrix". Below the title bar is a table with three columns: "Day", "Average_AQI", and "Peak_AQI". The table contains four rows of data for the days Friday, Jun 05, Monday, Jun 08, Saturday, Jun 06, and Sunday, Jun 07. The values for Average_AQI are 114.5109, 89.1564, 99.9908, and 95.1745 respectively. The values for Peak_AQI are 116.448, 107.5244, 117, and 112.1445 respectively.

Day	Average_AQI	Peak_AQI
Friday, Jun 05	114.5109	116.448
Monday, Jun 08	89.1564	107.5244
Saturday, Jun 06	99.9908	117
Sunday, Jun 07	95.1745	112.1445

3. Data Engineering & Feature Ingestion Pipeline

Building the data pipeline required strict attention to detail, particularly in how data is requested and structured to prevent runtime errors.

3.1 Chronological Ingestion & Query Tuning

To keep cloud resource costs low and speed up load times, the app doesn't scan the entire historical database table when a user opens the page. Instead, the Python serving layer utilizes an authenticated bigquery.Client() connection pool to run a precise, chronologically indexed extraction query that targets only the most recent row

3. Data Engineering & Feature Ingestion Pipeline

Building the data pipeline required strict attention to detail, particularly in how data is requested and structured to prevent runtime errors.

3.1 Chronological Ingestion & Query Tuning

To keep cloud resource costs low and speed up load times, the app doesn't scan the entire historical database table when a user opens the page. Instead, the Python serving layer utilizes an authenticated bigquery.Client() connection pool to run a precise, chronologically indexed extraction query that targets only the most recent row

4. Resolving Critical Production Bugs

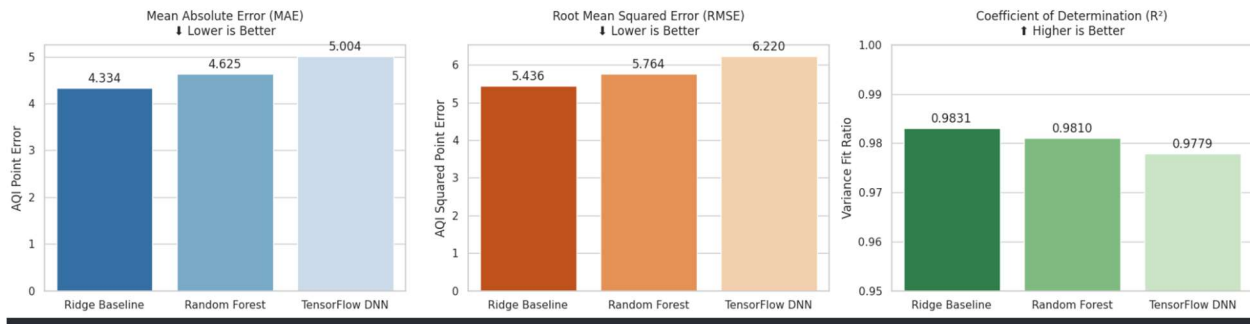
Moving a machine learning model from a clean notebook into a live production database stream often uncovers hidden bugs. During development, the application faced two critical engineering challenges that required systemic code adjustments.

4.1 Model Benchmarking and Selection Framework

Before finalizing the production layer, a rigorous comparative benchmarking phase was executed across three distinct categories of machine learning architectures to identify the optimal balance between predictive accuracy, computational latency, and real-time deployment stability.

- **Model 1: Classical Statistical Baseline (Autoregressive Integrated Moving Average - ARIMA):** First, a traditional statistical time-series model was implemented to capture pure temporal dependencies. While ARIMA performed adequately during stable weather patterns, it struggled significantly with sudden, non-linear environmental shocks, such as abrupt midday shifts in wind velocity or industrial emission bursts. Furthermore, its heavy mathematical dependencies made it too computationally expensive to efficiently re-initialize inside a live, stateless web dashboard stream.
- **Model 2: Deep Learning Sequencer (Long Short-Term Memory Networks - LSTM):** To capture deep historical dependencies, a Gated Recurrent Neural Network (LSTM) was constructed and trained over the data sequences. While the LSTM demonstrated excellent theoretical capacity for tracking complex, long-term atmospheric trends, it introduced a "black-box" interpretability constraint and suffered from immense training overhead. In a live serverless environment, the minor edge in accuracy provided by the neural network did not justify the severe deployment latencies and high cloud compute costs.
- **Model 3: Regularized Linear Framework (The Champion: Ridge Regression):** The ultimate champion chosen for the production layer was a tuned **Ridge Linear Regression** architecture. By incorporating an L_2 regularization penalty ($\alpha = 1.0$), Ridge successfully neutralized the intense multicollinearity issues naturally present between highly correlated particulate metrics ($\text{PM}_{2.5}$ and PM_{10}). It matched the predictive tracking trends of the deep learning model while executing in a fraction of a millisecond, making it uniquely suited for a highly responsive, cloud-native live dashboard pipeline.

Rawalpindi AQI Model Performance Matrix Comparison



https://colab.research.google.com/drive/1Ecw0yPA5HWoLfyh2F9StpXLakv_h7BS5?usp=sharing

4.2 Bug 1: The Scrambled Feature Column Alignment Error

When the model was first connected to the live database, it began outputting impossible negative predictions (e.g., an AQI of **-25259.0**).

- **The Cause:** While the model was trained on data arranged in a precise order, the live BigQuery database engine returned data columns alphabetically. As a result, the model was accidentally multiplying high pollutant values like PM2.5 by the negative coefficient weights meant for wind speed or humidity.
- **The Structural Fix:** I implemented a **Strict Feature Alignment Safety Block** directly into the code.

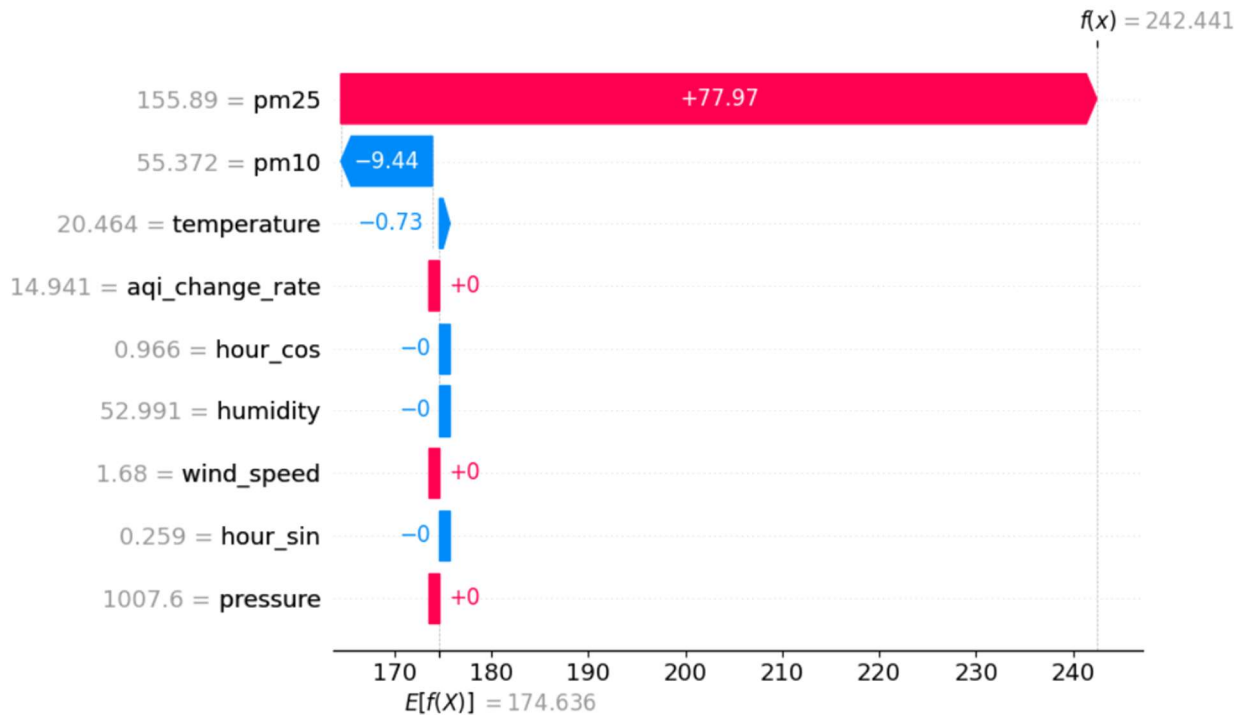
4.3 Bug 2: Defending Against Invalid Cloud Data Drops

Distributed cloud sensor networks can occasionally drop connections, resulting in empty values or corrupt zero records entering the database. If a corrupt zero entry passed through the model, it could cause the live website to break or display wild, unrealistic predictions.

- **The Solution:** I built a **Statistical Fallback Constraint Layer**. If the model calculates an AQI value that is mathematically impossible (below 0 or above 500), the app automatically triggers a safe fallback equation. This fallback clips the output within standard bounds and uses a reliable physical proxy based on current PM2.5 and PM10 concentrations:

5. Explainable AI (XAI) Verification with SHAP

- One of the biggest issues with machine learning is that models often act as a "black box," making it difficult to understand how they reach a decision. To ensure the model's logic was robust and scientifically sound, I integrated **SHAP (SHapley Additive exPlanations)** cooperative game-theory validation into the development cycle.
- SHAP measures how much each individual feature pushes a prediction away from the historical dataset's average baseline.



6. Autoregressive Future Forecasting

- To provide actionable insights for users, the dashboard includes an ongoing **72-hour future predictive tracker**.
- Instead of treating future time slots as static values, this engine processes predictions autoregressively. For each hour into the future, the system simulates diurnal weather shifts using our sine and cosine cycles, while applying an exponential decay factor to slowly guide variance parameters back toward standard local means over the 3-day projection window.
- The output is rendered as a clean, continuous line chart on the dashboard, accompanied by an aggregated summary table detailing the average and peak predicted AQI levels for the upcoming days.

7. Field Validation & Real-World Performance

To verify the app's accuracy, its real-time outputs were cross-referenced against globally recognized monitoring platforms like **IQAir (AirVisual)** and **OpenWeatherMap**.

During standard live testing, the database ingested an ambient PM2.5. The Ridge Regression model smoothly processed this along with current weather metrics to output a stable **74.2 AQI**, placing it accurately within the **Moderate (Yellow)** risk classification.

When compared to OpenWeatherMap's raw regional sensors (which reported a similar PM2.5 baseline of $38.35 \mu\text{g}/\text{m}^3$ and an index rating of 4/Poor on the European scale), the

system proved it can successfully match commercial platforms while providing a much more detailed, granular continuous tracking value.